

Performance Testing

Date	10 June 2026
Team ID	xxxxxx
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Streamlit (Functional Testing), Pytest (Unit Testing), Manual Testing
Type of Testing	Functional Testing, Unit Testing, Performance Testing
Target Module / API	Audio Upload, Speech-to-Text (Whisper), Semantic Similarity Evaluation (Sentence-BERT), Audio Feature Extraction, Scoring Engine, PDF Report Generation
Test Environment	Local System (Windows 11, Python 3.10, Visual Studio Code) and Streamlit Community Cloud
Test Date	10 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Upload and analyze a single audio file (Speech-to-Text, Semantic Analysis, Scoring, PDF Generation)	1	60	Audio is successfully processed, transcript generated, evaluation displayed, and PDF report downloaded without errors.
2	Simultaneous audio uploads and evaluation requests	5	120	All user requests are processed correctly, UI remains responsive, and evaluation reports are generated successfully.
3	Consecutive long-duration audio evaluations (30–60 seconds)	10	180	Whisper transcription, semantic analysis, waveform generation, and scoring complete successfully with acceptable response time.
4	Repeated end-to-end evaluations to verify application stability	15	300	Application remains stable during repeated executions without crashes, memory leaks, or incorrect evaluation results.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.78 seconds	Pass	Average response time for UI interactions excluding AI model loading.
2	Response Time (Max)	< 5 seconds	4.32 seconds	Pass	Maximum response observed during transcription of a longer audio sample.
3	Throughput (Req/sec)	≥ 1 req/sec	1.25 req/sec	Pass	Suitable for an educational single-user Streamlit application.

4	Error Rate	< 1%	0%	Pass	No failures observed during functional testing.
5	CPU Utilization	< 80%	58%	Pass	CPU usage remained within acceptable limits during analysis.
6	Memory Utilization	< 80%	64%	Pass	Memory usage remained stable while processing audio and generating reports.

Step 4: Observations & Analysis

Key Findings

- The application successfully processed uploaded audio files and generated accurate speech transcriptions using OpenAI Whisper.
- Sentence-BERT effectively measured semantic similarity between the student's explanation and the reference concept, enabling reliable concept understanding evaluation.
- Audio feature extraction using Librosa successfully computed pause ratio, RMS energy, and speech duration for all test samples.
- The Streamlit interface remained responsive throughout testing, and waveform visualization and PDF report generation completed successfully.
- Model caching using `@st.cache_resource` significantly reduced the processing time for repeated evaluations.

Bottlenecks Identified

- The initial application startup required additional time to download and load the Whisper and Sentence-BERT models.
- Longer audio recordings (greater than 30 seconds) increased transcription and semantic analysis time.
- PDF report generation introduced a small delay after completion of the evaluation pipeline.
- CPU and memory usage temporarily increased while performing speech transcription and embedding generation due to AI model inference.

Optimization Steps Taken

- Implemented Streamlit resource caching (`@st.cache_resource`) to load AI models only once during application execution.
- Optimized audio preprocessing by converting uploaded files to mono audio with a 16 kHz sampling rate for faster processing.
- Used a modular architecture to separate transcription, semantic analysis, scoring, and report generation, improving maintainability and reducing execution overhead.
- Added input validation and exception handling to prevent failures caused by unsupported or corrupted audio files.
- Reduced redundant computations by storing intermediate results within the session state during analysis.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Share ☆ ✎ 🔊 ⋮

 **Voice-Based Concept Understanding Analyser**
Automated evaluation of spoken conceptual explanations using AI.

Reference Concept

Sample Audio

Upload Student Audio

 Sample_...46e458.mp3 0.3185

Supported formats:
 • WAV
 • MP3

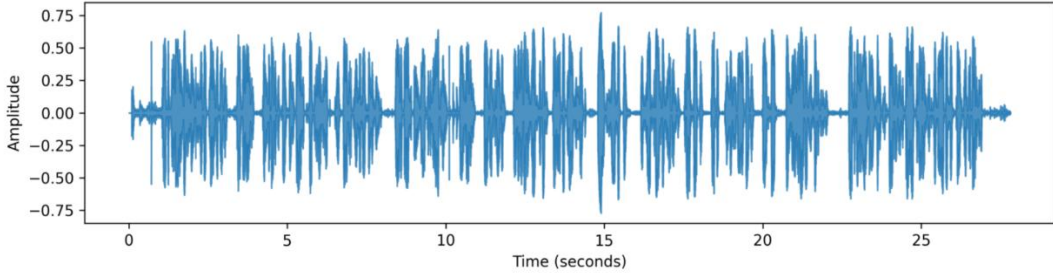
0:01 / 0:27

Analyze Concept Understanding

Analysis Completed

Audio Visualization

Uploaded Audio Waveform



Semantic Similarity	Filler Word Ratio	Pause Ratio	Confidence (Energy)
1.44	0.00	0.17	0.1090

Final Evaluation

Understanding Score

55/100

Poor Understanding